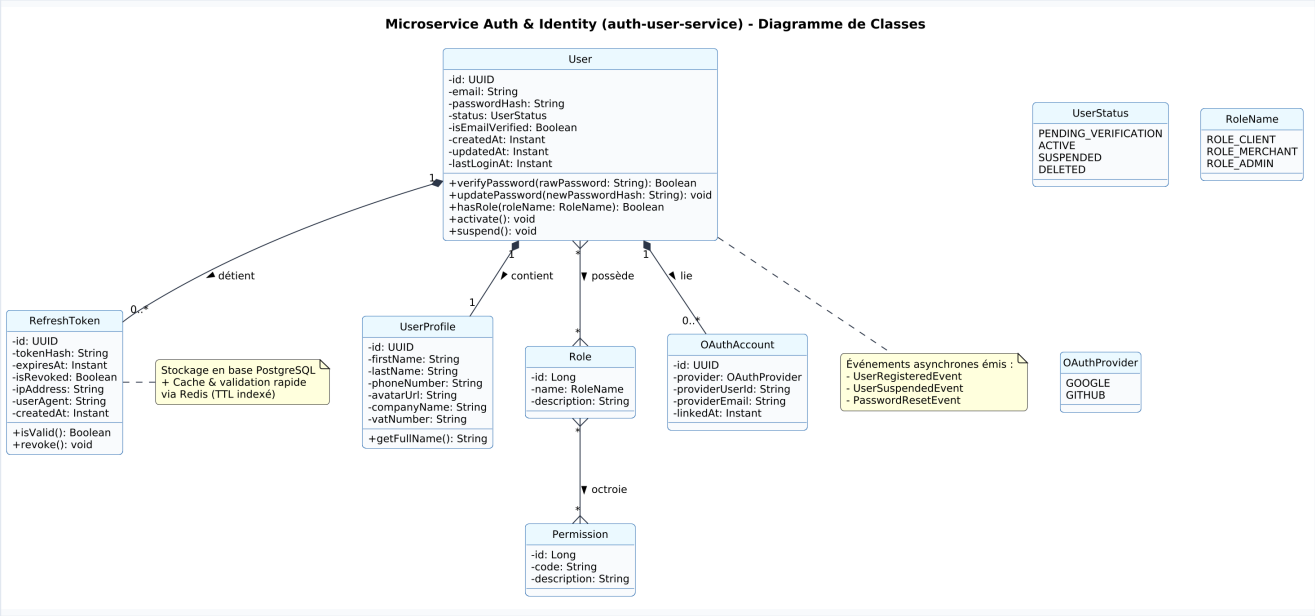
Partie Individuelle : Identity & Access Service (IAM)

Membre 1 — Responsable Authentification, Gestion des Utilisateurs & Autorisations

Speech de Présentation (1 minute chrono) :
*« Bonjour, je suis responsable du microservice **Identity & Access Service**, garant de la sécurité et des accès d'OmniShop. Notre mission couvre l'inscription/connexion locale et fédérée via **OAuth2/OpenID Connect** (Google, GitHub), la gestion des profils, et un cloisonnement strict par **RBAC** entre acheteurs débutants et marchands professionnels. Nous combinons des **JWT asymétriques RS256** (stateless pour l'API Gateway via **JWKS**) et des **Refresh Tokens sous Redis** pour une révocation instantanée. Enfin, nous émettons des événements asynchrones sur **RabbitMQ** comme UserRegisteredEvent pour déclencher sans couplage la création de boutique côté marchand. »*

1. Conception du Microservice (Diagramme de Classes UML)

Modélisation du modèle relationnel et sécuritaire du microservice sous PostgreSQL (intégrité ACID) et Redis (cache & sessions) :

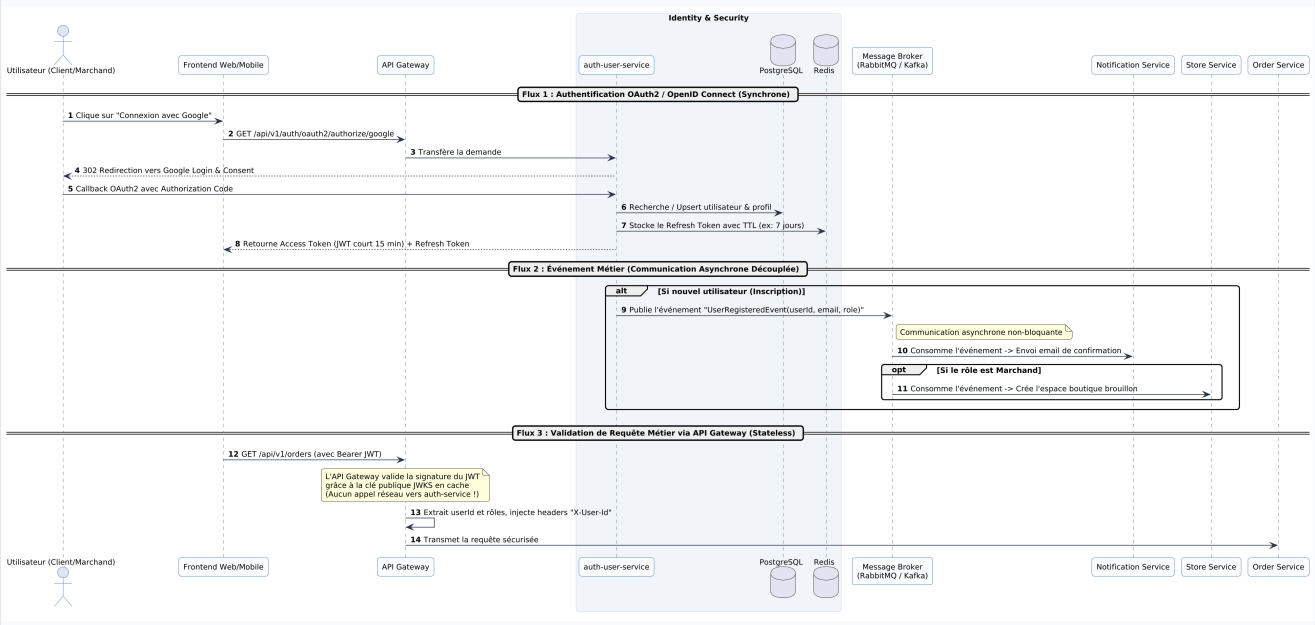


Détail des Entités & Responsabilités Métier :

- **User :** Racine d'agrégat (UUID, email unique vérifié, hachage de mot de passe Argon2/BCrypt, statut de compte).
- **Role & Permission (RBAC) :** Cloisonnement granulaire (ROLE_CLIENT, ROLE_MERCHANT, ROLE_ADMIN). Un client accède uniquement à ses commandes ; un marchand administre son catalogue et ses tarifs B2B.
- **OAuthAccount :** Liaison de comptes externes (fournisseur Google/GitHub, identifiant distant, synchro de profil).
- **RefreshToken :** Jeton persistant chiffré stocké sous Redis avec TTL (7j) et détection de rejeu / vol de session.

2. Interactions avec les Autres Microservices

Ce diagramme de séquence illustre la coexistence de communications **synchrones** (OAuth2, JWKS) et **asynchrones** (Message Broker) :



Exemples de Communications :

• **Communication Synchrone (API Gateway & JWKS) :** L'API Gateway interroge au démarrage GET /.well-known/jwks.json pour mettre en cache la clé publique RSA. Elle vérifie ainsi la signature des requêtes entrantes de façon 100% autonome et *stateless* sans surcharger le microservice d'authentification.

• **Communication Asynchrone (RabbitMQ) :** Lors d'une inscription, auth-user-service publie l'événement UserRegisteredEvent. Le notification-service consomme l'événement pour expédier le courriel de confirmation, et le store-service instancie automatiquement l'espace marchand brouillon.

3. Contrats d'API & Spécifications des Endpoints

Méthode & URI	Description Métier	Payload / Paramètres	Codes de Réponse	Événement Broker
POST /api/v1/auth/register	Inscription nouvel utilisateur	email, password, role, firstName, lastName	**201** Created **400** Bad Request **409** Conflict (Email pris)	UserRegisteredEvent
POST /api/v1/auth/login	Authentification locale	email, password	**200** OK (Access+Refresh Tokens) **401** Unauthorized **403** Compte suspendu	UserLoggedInEvent
POST /api/v1/auth/refresh	Renouvellement token expiré	refreshToken	**200** OK (Nouveau couple JWT) **401** Unauthorized (Jeton révoqué)	—
GET /api/v1/auth/oauth2/authorize/{p}	Lancement flux OAuth2 (Google/GitHub)	provider = google | github	**302** Found (Redirection vers mire de login tierce)	—
GET /api/v1/auth/oauth2/callback/{p}	Callback OAuth2 et échange token	code, state	**200** OK (Tokens émis) **400** Code invalide	UserRegisteredEvent (si 1ère fois)
POST /api/v1/auth/logout	Déconnexion & révocation	refreshToken (Bearer JWT)	**204** No Content **401** Non authentifié	—
GET /.well-known/jwks.json	Diffusion clés publiques RSA	Aucun (Public)	**200** OK (JSON Web Key Set)	—